

MISIÓN: *El Quijote.js* – La venganza de los bugs

Contexto narrativo

En un lugar de Node.js, de cuyo nombre no quiero acordarme...
un grupo de valientes desarrolladores debe analizar el texto del
Don Quijote de la Mancha
para liberar al código de los terribles **bugs gigantes (que en realidad son logs)**.

OBJETIVO

Construir un sistema modular en Node.js que analice el Quijote como si fuera...
un dungeon lleno de strings salvajes 🐉

REGLAS DEL JUEGO (IMPORTANTE)

 El sabio Cervantes exige:

- ❌ NO todo en un solo archivo (herejía)
 - ✅ Mínimo 3 módulos (`reader`, `analyzer`, etc.)
 - ✅ Uso de `module.exports`
 - ✅ Código limpio (o serás desterrado a callback hell)
-



NIVELES DE LA AVENTURA



NIVEL 1: *El despertar del becario*

💬 “Sancho... léeme ese archivo”



Misiones:

1. Leer el archivo `quijote.txt` usando `fs`
2. Mostrar cuántos caracteres tiene



Pregunta trampa:

¿Qué pasa si usas `readFileSync` en un servidor con 1000 usuarios?

(Spoiler: 🧠)



NIVEL 2: *Los molinos de palabras*

💬 “No son molinos... ¡son strings mal parseados!”



Misiones:

3. Contar palabras
4. Mostrar las 10 más repetidas



Extra:

- Ignorar palabras como: el, la, de, que...



Pregunta:

¿Por qué `"Hola, "` y `"Hola"` son diferentes si no limpias el texto?

NIVEL 3: *Sanitizando el caos*

 “Sancho... esto tiene más comas que un JSON roto”

Misiones:

5. Limpiar el texto:
 - minúsculas
 - sin puntuación
6. Crear función reutilizable en `utils.js`

Pregunta:

¿Por qué es mala idea repetir lógica en varios archivos?

NIVEL 4: *El ataque de los módulos*

 “Divide y vencerás... sobre todo si es código”

Misiones:

7. Separar en módulos:
 - `reader.js`
 - `analyzer.js`
 - `utils.js`
8. Exportar funciones correctamente

Pregunta:

¿Qué diferencia hay entre exportar una función y un objeto?

NIVEL 5: *El jefe final: Sancho.exe*

 “Señor... creo que ese while es infinito...”

Misiones:

9. Buscar cuántas veces aparece:
 - "quijote"
 - "sancho"
10. Calcular:
 - longitud media de palabras

🟪 NIVEL LEGENDARIO: *Modo hacker del siglo XVII*

💬 “He aquí el verdadero poder... process.argv”

🔧 Misiones:

11. Ejecutar el programa así:

```
node app.js --top=20
node app.js --buscar=quijote
```

12. Guardar resultados en `resultados.txt`

💀 PENALIZACIONES (porque sí)

- ❌ Código duplicado → -10 honor points
 - ❌ Todo en `app.js` → destierro inmediato
 - ❌ Variables tipo `x`, `y`, `cosa` → cárcel semántica
-

🏆 BONUS (para alumnos que quieren romper el juego)

💡 Implementar:

- Lectura con **streams**
 - Ranking de palabras más largas
 - Detector de palabras épicas tipo:
 - “caballero”
 - “aventura”
 - “desventura” 😞
-

🧠 PREGUNTAS FILOSÓFICAS (modo profesor serio pero no mucho)

- ¿Por qué Node.js es bueno para procesamiento de texto?
 - ¿Qué ventajas tiene modularizar?
 - ¿Qué pasaría si el archivo pesa 2GB?
 - ¿Por qué limpiar datos es el 80% del trabajo real?
-



FRASE FINAL (para cerrar la clase)

“No luches contra molinos...
lucha contra funciones mal diseñadas.”